

COMP8260 PROJECT FOR GROUP P08: Classifying melanoma images using a CNN

Uyi Owen-Ode, 23000585, uno3@kent.ac.uk
Henry Ward, 17882778, hw328@kent.ac.uk
Gayathri Kumar, 24010288, gk329@kent.ac.uk
Bhawish Raj br323, ,
Christopher Bello, 21036354, tb647@kent.ac.uk

,

Make sure the report contains a link to your project repository on <https://git.cs.kent.ac.uk/> , and make sure to give access the project to users mm53, grba, mb2076, mjb226, rh667 and zl249 with "Reporter" role (see <https://docs.gitlab.com/ee/user/project/members/#add-users-to-a-project>)

FROM PROJECT BRIEF:

1x Report (Research, Design, References) You are to hand up a report as a group (**max 15 pages**) excluding references. Your report should include but not be limited to: rationale, architectural design, implementation, performance evaluation including runtime performance, project management, development flow. The report should include a Cover page with each member Student ID, Name and Email. References should adopt the Harvard Style.

COMP8260 PROJECT FOR GROUP P08: Classifying melanoma images using a CNN

Rationale

In the realm of medical imaging and diagnostics, the early and accurate detection of melanoma, a type of skin cancer, is critical for effective treatment and patient survival (Robinson *et al.*, 2016; Forschner *et al.*, 2018). Traditional methods of diagnosis rely on dermatologists' visual assessment of skin lesions, which, while effective, can vary in accuracy due to factors such as the dermatologist's experience level and the subjective nature of visual examination (Roetzheim *et al.*, 2000). With the advent of machine learning and deep learning technologies, there has been a significant shift towards automating and enhancing the accuracy and efficiency of such diagnostic processes (Stiff *et al.*, 2022). Among the myriad of algorithms available, Convolutional Neural Networks (CNNs) have emerged as particularly powerful tools for image classification tasks, including the differentiation of benign and malignant melanomas (Naeem *et al.*, 2020). In this report we describe the use of a CNN for classifying images of melanomas due to their ability to learn hierarchical feature representations from raw pixel data (Nasr-Esfahani *et al.*, 2016), their superior accuracy compared to traditional image processing and machine learning techniques (Nasr-Esfahani *et al.*, 2016; Cao *et al.*, 2021), and their potential to augment dermatological expertise by providing a consistent and scalable diagnostic tool. Through the leveraging of CNNs, the medical community can make strides towards more reliable, objective, and accessible melanoma screenings, ultimately improving patient outcomes and advancing the frontier of dermatological diagnostics.

Architectural Design and Performance Evaluation

This section provides a detailed overview of the architectural design of different Convolutional Neural Networks (CNN) that we developed tailored for classifying melanoma images into benign or malignant and the model performance on chosen metrics. It explains the underlying rationale for the architecture, highlights its critical components, and outlines the layer-wise configuration, emphasizing how the network discerns between benign and malignant melanomas through image analysis as well as performance evaluation for all the models we developed.

It also details the methodology used for testing, the metrics for evaluation, and the results obtained, providing a critical analysis of the model's effectiveness and areas for improvement.

Architectural Overview

The architecture of the CNN is crafted to efficiently process dermatological images, extracting and learning intricate features to accurately classify melanomas. It consists of several layers, each playing a pivotal role in the feature extraction and learning mechanism. The CNN

architecture comprises multiple convolutional layers followed by max-pooling layers, designed to extract hierarchical features from input images. The convolutional layers employ rectified linear unit (ReLU) activation functions to introduce non-linearity while max-pooling layers reduce spatial dimensions and retain salient features. The fully connected layers further process the extracted features before producing the final binary classification output using a sigmoid activation function.

Data Preprocessing:

Before training the CNN model, the dataset underwent some preprocessing steps to enhance model performance. The images in the dataset were already resized to a standardized format of 224x224 pixels. However, normalization of pixel values to a range suitable for neural network training was carried out before feeding into the model. These preprocessing techniques help mitigate overfitting by decreasing the effect of noise during training and improving the model's ability to generalize to unseen data.

Convolutional Layers: Fundamental to the CNN, these layers extract features from the input images using filters (or kernels), generating feature maps that capture essential patterns such as edges and textures. We experimented with different numbers of convolutional layers with an increasing number of filters in each subsequent convolutional layer to allow the network to capture a broader range of features at each hierarchical level.

Activation Functions: Following each convolutional layer, non-linear activation functions (e.g., ReLU) are applied to introduce non-linearity, enabling the network to learn complex data patterns. ReLU is also helpful in avoiding the vanishing gradient problem during the training.

Pooling Layers: After each convolutional layer, a MaxPooling layer has been used as it helps in achieving translation invariance, which means the model can recognize the features regardless of their position in the input image. By selecting the maximum value within each pooling window, MaxPooling retains the most salient features while discarding irrelevant details. Moreover, MaxPooling acts as a form of regularization and helps prevent overfitting by reducing the model's sensitivity to small variations in the input data and promoting spatial generalization.

Fully Connected Layers: These layers integrate learned features from preceding layers, facilitating high-level reasoning and the beginning of the classification process. In our model, three dense layers have been used with the ReLU activation.

Output Layer: The output layer consists of only one unit and a sigmoid activation function, outputting probabilities for the image classifications (benign or malignant), with the highest probability determining the model's prediction.

- The optimization algorithm used for training included stochastic gradient descent, Adam, RMSProp and it was found that Adam gave us better results in terms of higher training and test accuracy for all of the CNNs.
- Moreover, 5-fold cross-validation has been used to better train the model through training/validation splits to avoid overfitting.
- Stride is by default 1x1. Higher values of stride such as 2x2 did not lead to a decrease in training time nor did it have any effect on accuracy.
- Padding has not been used in the model as for cancer melanoma classification tasks specifically, padding may not be required as the input images are preprocessed to have consistent sizes and aspect ratios before being fed into the neural network so padding might not be needed to maintain spatial consistency.
- Techniques like early stopping, dropout and L1L2 regularization have been employed to prevent overfitting of the models.

Evaluation Metrics:

Confusion matrix, accuracy, precision, recall, F1-score were the evaluation metrics used in the model whereas binary cross entropy was used as a loss function. These evaluation metrics offer a comprehensive assessment of the model's performance in melanoma cancer classification, considering factors such as overall accuracy, ability to correctly identify malignant lesions (precision), sensitivity to detecting malignant cases (recall), and a balanced measure of precision and recall (F1-score). This holistic evaluation helps gauge the model's predictive performance as well as utilization in melanoma diagnosis and treatment decisions.

First CNN Model:

We started developing our first CNN model by keeping it simple with the aim of evaluating various optimizers to zero in on the optimal choice of the optimizer.

Layer-wise Configuration

Below is the layer-wise configuration of our designed CNN model used for melanoma cancer images classification.

1. Input Layer:

- The input layer expects input images with shape (224, 224, 3) for width, height, and three colour channels for RGB primary colours. The input layer remains the same throughout all of our models discussed in the report.

2. Convolutional Layers:

This model consists of two Conv2D layers with 32 and 64 filters respectively, 3x3 kernel size, valid padding, and ReLU activation. A default stride of 1x1 was used in the model. MaxPooling2D layer with a 2x2 pool size was added after each convolutional layer.

3. Flatten Layer:

A flattening layer was added to flatten the multidimensional output from the convolutional layers into a 1D vector.

4. Fully Connected Layers:

No fully connected layer was used in this model besides the output layer.

5. Output Layer:

Dense layer with 1 unit, sigmoid activation for binary classification.

6. Compilation:

- We tried three different optimizers for training our model including adam, RMSProp and SGD.
- The loss function used was binary_crossentropy because of the binary classification problem
- The Metrics used for performance evaluation is binary_accuracy
- The model was trained with a default batch size of 32 and for 10 epochs.

Performance of the Model:

The following performance was achieved by training this model on different optimizers.

Performance Evaluation			
	adam	RMSProp	SGD
Training Accuracy	81.5%	80.54%	58.82%
Test Accuracy	80.5%	82.5%	50%
Loss	0.427	0.4	0.7

Based on the above metrics, it can be said that our simple CNN model was able to provide us with an acceptable level of training and test accuracy and adam as an optimizer works well compared to the other two optimizers. In RMSProp, the model seems to be underfitting as it has

not learned the training data well but can generalize better to test data. Lastly, SGD appears to be performing poorly on this dataset.

Second CNN Model:

We decided to build onto the first CNN through more layers and aim of evaluating various optimizers in order to zero in on the optimal choice of the optimizer.

Layer-wise Configuration

Below is the layer-wise configuration of our designed CNN model used for melanoma cancer images classification.

1. Input Layer:

- Same for all Models

2. Convolutional Layers:

This model consists of four Conv2D layers with the first layer having 32 filters and the rest three convolution layers having 64 filters, 3x3 kernel size, valid padding, and ReLU activation. A default stride of 1x1 was used in the model. MaxPooling2D layer with a 2x2 pool size was added after each convolutional layer.

3. Flatten Layer:

A flattening layer was added to flatten the multidimensional output from the convolutional layers into a 1D vector.

4. Fully Connected Layers:

No fully connected layer was used in this model besides the output layer.

5. Output Layer:

Dense layer with 1 unit, sigmoid activation for binary classification.

6. Compilation:

- We tried three different optimizers for training our model including Adam, RMSProp and SGD.
- The loss function used was `binary_crossentropy` because of the binary classification problem

- The Metric used for performance evaluation is accuracy
- The model was trained with a default batch size of 32 and for 55 epochs.

Performance of the Model:

The following performance was achieved by training this model on different optimizers.

Performance Evaluation			
	Adam	RMSProp	SGD
Training Accuracy	88.55%	88.8%	85.48%
Test Accuracy	85.65%	89.25%	86.8%
Loss	32.8%	29.03%	32.65%

Based on the above metrics, it can be said that our simple CNN model was able to provide us with a more improved level of training and test accuracy and RMSProp as an optimizer works well compared to the other two optimizers. SGD although shows an improvement, is still not up to mark.

Third CNN Model:

We decided to add further through more layers and aim to evaluate various optimizers to zero in on the optimal choice of the optimizer.

Layer-wise Configuration

Below is the layer-wise configuration of our designed CNN model used for melanoma cancer images classification.

1. Input Layer:

The input layer remains the same as for our previous models discussed in the report.

2. Convolutional Layers:

This model consists of six Conv2D layers with one 32, three 64 and two 128 filters, 3x3 kernel size, valid padding, and ReLU activation. A default stride of 1x1 was used in the model. MaxPooling2D layer with a 2x2 pool size was added after each convolutional layer.

3. Flatten Layer:

A flattening layer was added to flatten the multidimensional output from the convolutional layers into a 1D vector.

4. Fully Connected Layers:

No fully connected layer was used in this model besides the output layer.

5. Output Layer:

Dense layer with 1 unit, sigmoid activation for binary classification.

6. Compilation:

- We tried two different optimizers for training our model including Adam, and RMSProp.
- The loss function used was `binary_crossentropy` because of the binary classification problem
- The Metrics used for performance evaluation is accuracy
- The model was trained with a default batch size of 32 and for 10 epochs.

Performance of the Model:

The following performance was achieved by training this model on different optimizers.

Performance Evaluation			
	Adam	RMSProp	SGD
Training Accuracy	87.02%	86.78%	NA
Test Accuracy	82.4%	84.9%	NA

Loss	42.06%	35.31%	NA
-------------	--------	--------	----

Based on the above metrics, it can be said that this 6 layer simple CNN model while performing well is not as good as that of the 4 layer one. The accuracies are slightly lesser and the loss is a bit more significant here. SGD was dropped as it is not improving the models' performance.

Fifth CNN Model:

Building upon previously developed models, the configuration of fifth model is as follows:

Layer-wise Configuration

Below is the layer-wise configuration of our designed CNN model used for melanoma cancer images classification.

1. Input Layer:

The input layer remains the same as for our previous models discussed in the report.

2. Convolutional Layers (6x):

- **Block 1:**
 - One Conv2D layer with 32 filters, 3x3 kernel size, valid padding, and ReLU activation.
 - MaxPooling2D layer with 2x2 pool size after each Conv2D layer.
- **Block 2:**
 - Three Conv2D layers with 64 filters, 3x3 kernel size, valid padding, and ReLU activation.
 - MaxPooling2D layer with 2x2 pool size after each Conv2D layer.
- **Block 3:**
 - Two Conv2D layers with 128 filters, 3x3 kernel size, valid padding, and ReLU activation.
 - MaxPooling2D layers with 2x2 pool size after each Conv2D layer.

3. Flatten Layer:

A flattening layer was added to flatten the multidimensional output from the convolutional layers into a 1D vector.

4. Fully Connected Layers (3x):

The dense layer consists of 64 units, ReLU activation, and L1L2 regularization ($l1=0.01$, $l2=0.001$).

5. Output Layer:

The output layer consists of a dense layer with 1 unit, sigmoid activation for binary classification.

6. Compilation:

- Cross-validation technique with 3-folds has been used for assessing and improving the performance and generalization ability of the model
- The final model was trained only on adam optimizer as it gave optimal results throughout all of the models
- The loss function used was binary_crossentropy because of the binary classification problem
- The Metrics used for performance evaluation is binary_accuracy
- The model was trained with a default batch size of 32 and for 10 epochs.

Performance of the Model:

The following performance was achieved by training this model on different optimizers.

Performance Evaluation	
Training Accuracy	%
Test Accuracy	89.65%
Loss	0.28

Based on the above metrics, it can be said that this CNN model was able to provide us with higher train and test accuracy by making a complex model with an increased number of convolutional layers and employing different techniques such as cross-validation, L1L2 regularization.

Summary of all Trained Models

S no.	Model Code	Num of Epochs	K fold Cross Validation	No. of Conv2D layers	Train Accuracy	Test Accuracy	% Overfitting
1	Adam02	10	No	2	81.50%	80.50%	1.00%
2	Rmsprop02	10	No	2	80.54%	82.50%	-1.96%
3	SGD02	10	No	2	58.82%	50.00%	8.82%
4	Adam04	10	Yes	4	88.55%	85.65%	2.90%
5	Rmsprop04	10	Yes	4	88.80%	89.25%	-0.45%
6	SGD04	10	Yes	4	85.48%	86.80%	-1.32%
7	Rmsprop06	20	Yes	6	87.02%	82.40%	4.62%
8	Adam06	20	Yes	6	86.78%	84.90%	1.88%
9	Adam0601	30	Yes	6	86.76%	89.65%	-2.89%
10	Adam0602	20	Yes	6	86.76%	88.65%	-1.89%

Implementation (Uyi)

Development Environment Setup

Hardware Requirements: The implementation was carried out on the University of Kent Jupyter server and on Google colab.

Software: The project utilized Python Programming Language , with key libraries including TensorFlow ,Keras,matplotlib,numpy,os,PIL,Sklearn version .

Data Preparation

Dataset - The model was trained on a dataset comprising 11,879 train images and 2,000 test images with their respective labels , with a validation split of 20%.

The train images are made up of 6,289 malignant cancer labeled images and 5,590 Bening cancer labeled images making a 53% and 47% split respectively. The test images are made up of 2000 images with a 50% split between both labels. For training, the simpler models at the beginning of the project utilized a validation split of 20% split of the training data for validation . K fold cross validation was used for the complex models towards the end of the project.

Preprocessing Steps - :We used images with dimension (224,224,3) Image normalization was carried out by dividing the images by 255 after their data type was changed to float32 .

Training Process

Optimization Algorithm - Three optimizers, namely rmsprop,SGD and Adam were used for the simpler models at the beginning of the project in order to determine an optimal model for the best accuracy.

Batch Size and Epochs -: The model was trained with a batch size of 32 and 128 over 10,20 and 30 epochs for the various models that were trained.

Performance Monitoring - Training progress was monitored using accuracy to adjust parameters and improve model performance.

Challenges Encountered

Some team members had problems accessing the University of Kent jupyter website for an extended period due to the long training duration required for this project. Others had problems installing dependencies when training or testing the model from within different Jupyter notebook environments. This is a known issue stemming from the abstraction of Jupyter notebooks (*Installing Python Packages from a Jupyter Notebook* | *Pythonic Perambulations*, no date).

Future Directions

Model Improvement - Explore advanced neural network architectures and training techniques to further enhance model accuracy.

Dataset Expansion - Increase the dataset size and diversity to improve the model's robustness and generalizability.

Data Augmentation - In order to improve the quality of data set hence generalization,Data Augmentation could be carried out to achieve this.

Clinical Integration - Work towards integrating the CNN into clinical workflows to assess its real-world effectiveness and usability.

Recommendations

- **Model Refinement**: [Suggestions for Model Tuning, Additional Layers, or Architectural Changes].
- **Data Augmentation**: Exploring more sophisticated data augmentation techniques to improve model generalization.
- **Transfer Learning**: Investigating the use of pre-trained models to enhance feature extraction capabilities.
- **Clinical Integration**: Assessing the feasibility and impact of integrating the CNN into clinical workflows for real-world validation. A 2020 systematic review of CNNs used for melanoma classification found that they performed poorly for dark-skinned people (Naeem *et al.*, 2020) However, by 2023 a group of researchers in Nigeria used a transfer learning process to create a model that classified melanomas on dark skin with 98% accuracy (Kushimo *et al.*, 2023).

Project Management (Chris)

This section delves into the project management strategies and structures implemented to oversee the development and deployment of the Convolutional Neural Network (CNN) for classifying melanoma images. It outlines the organizational framework, roles and responsibilities, timelines, and tools utilized to ensure the project's objectives were met efficiently and effectively.

Project Framework

Melanoma, a type of skin cancer, poses a significant health risk if left undetected and untreated. Early and accurate detection is crucial for effective treatment and improved patient outcomes. Leveraging the power of deep learning, this project aimed to develop a Convolutional Neural Network (CNN) capable of accurately classifying melanoma images as benign or malignant, providing a valuable tool for healthcare professionals.

Project Scope

The project aimed to develop a CNN capable of accurately classifying melanoma images into benign or malignant categories. The scope included data collection, preprocessing, model development, testing, and deployment. The primary objective was to create a reliable and user-friendly tool that could assist healthcare professionals in the early detection and diagnosis of melanoma.

Methodology

An Agile/Scrum methodology was adopted to allow for flexibility in development and incorporate stakeholder feedback throughout the project lifecycle. This iterative approach enabled the team to adapt to changing requirements and evolving needs, ensuring that the final product met the stakeholders' expectations.

Team Structure

The project team consisted of five members, including a Project Manager, two Data Scientists, two Software Developers, two QA Engineers, and one UI/UX Designer. This diverse team brought together expertise in project management, data analysis, software development, quality assurance, and user experience design, ensuring a well-rounded and comprehensive approach to the project.

Roles and Responsibilities

Project Manager

The Project Manager oversaw project planning, coordination, and communication, ensuring that milestones were met and resources were allocated effectively. They facilitated regular meetings,

monitored progress, and addressed any issues or roadblocks that arose during the project's lifecycle.

Key Responsibilities

- Developing and maintaining the project plan
- Coordinating team activities and resources
- Facilitating communication among team members and stakeholders
- Monitoring project progress and addressing risks

Data Scientists

The Data Scientists were responsible for data collection, preprocessing, and model development, including tuning and validation. They ensured the quality and relevance of the data used for training the CNN model.

Key Responsibilities

- Collecting and curating melanoma image data
- Preprocessing and cleaning the data
- Developing and training the CNN model
- Evaluating and optimizing model performance

Software Developers

The Software Developers focused on integrating the CNN into the application framework, developing the user interface, and implementing backend services. They ensured that the CNN model was seamlessly integrated into a user-friendly application.

Key Responsibilities

- Integrating the CNN model into the application framework
- Developing the user interface for image upload and classification
- Implementing backend services for data handling and classification
- Ensuring application performance and scalability

Project Timeline

Initiation and Planning

The initiation and planning phase took place from [Start Date] to [End Date]. During this phase, the project charter was developed, stakeholder meetings were held, and resources were planned. The team established clear objectives, defined the scope, and created a detailed project plan.

Development Phases

The development phase was broken down into four sprints, each focusing on specific features or components, running from [Start Date] to [End Date]. The first sprint focused on data collection and preprocessing, the second sprint on model development and training, the third sprint on application development and integration, and the fourth sprint on testing and deployment.

Testing and Deployment

The testing and deployment phase occurred from [Start Date] to [End Date]. This phase included rigorous testing, including functional, performance, and security testing, to ensure the application met the highest quality standards. Once testing was completed, the application was deployed to production environments, making it available for use by healthcare professionals.

Tools and Technologies

Development Tools

The project utilized various development tools and frameworks for coding, version control, and continuous integration/continuous deployment (CI/CD). These included Python and its libraries (TensorFlow, Keras, OpenCV), Git for version control, and Jenkins for CI/CD.

Project Management Software

The team used Jira as the project management software, which facilitated task assignment, progress tracking, and communication among team members. Jira's agile boards and reporting features provided visibility into project progress and helped identify potential roadblocks.

Communication Tools

Effective communication was essential for the project's success. The team employed Teams, Zoom and Whatsapp for daily stand-ups, team meetings, and stakeholder updates. Teams and Zoom channels and integration with other tools ensured seamless collaboration and knowledge sharing.

Challenges and Solutions

Data Quality and Availability

One of the significant challenges faced during the project was obtaining high-quality and diverse melanoma image data. To address this challenge, the team collaborated with healthcare institutions and utilized publicly available datasets. Additionally, data augmentation techniques were employed to increase the diversity and quantity of training data.

Model Performance and Optimization

Achieving optimal model performance and accuracy was crucial for the project's success. The team tackled this challenge by exploring various CNN architectures, hyperparameter tuning, and employing techniques such as transfer learning and ensemble methods. Regular model evaluation and iterative refinement were key to improving performance.

Conclusion

The project management approach for the CNN for melanoma classification was designed to foster a collaborative and flexible development environment, enabling the team to adapt to challenges and incorporate feedback efficiently. Through careful planning, clear communication, and the judicious use of technology and methodologies, the project successfully met its objectives, delivering a tool that enhances the accuracy and efficiency of melanoma classification.

Recommendations for Future Projects

Enhanced Communication

While the project employed various communication tools, further improvements in communication channels between team members and stakeholders can help in early identification and resolution of issues. Implementing regular retrospective meetings and incorporating stakeholder feedback more frequently could lead to smoother project execution.

Agile Practices

The adoption of Agile practices, such as Scrum, proved beneficial for the project's flexibility and stakeholder satisfaction. Continued refinement and adherence to Agile principles, including regular iterations, retrospectives, and continuous improvement, can further enhance project outcomes.

Technology Upgrades

The field of artificial intelligence and machine learning is rapidly evolving. Keeping abreast of the latest developments in these technologies and exploring new frameworks, libraries, and techniques can lead to improved model performance, efficiency, and scalability. Continuously evaluating and incorporating new technological advancements should be a priority for future projects.

Development Flow

This section outlines the development flow of the Convolutional Neural Network (CNN) project for melanoma classification, emphasizing the adoption of Agile methodology. Agile's iterative and flexible approach was instrumental in managing the project's dynamic requirements, enhancing team collaboration, and ensuring the delivery of a high-quality software product. This narrative will describe the stages of the development process, the Agile practices employed, and how these practices facilitated the project's objectives.

Agile Methodology Overview

Agile methodology focuses on continuous delivery, customer feedback, and the ability to adapt to changing requirements. For this project, the Scrum framework guided the development process, characterized by the following key practices:

- **Sprint Planning:** Regular sprint planning sessions were held to prioritize tasks and set achievable goals for short development cycles (sprints).
- **Daily Stand-ups:** Daily meetings via WhatsApp helped the team synchronize their work and address any obstacles in the development process.
- **Iterative Development:** The project was divided into small, manageable increments, allowing for iterative development and frequent assessment.
- **Continuous Integration and Deployment (CI/CD):** Automated pipelines were not implemented to ensure continuous integration of code changes on this occasion but we feel that would be very effective MLOps for a future project, especially if aiming to deploy models into a production environment.

Phases of Development

1. Pre-Sprint Planning:

- **Goal Identification:** Initial project goals and high-level requirements were defined through team discussion and consultation with lecturers.
- **Backlog Creation:** A “product backlog” of potential features, enhancements, and foreseeable bugs was developed.

2. Sprint Execution:

- **Sprint 1:** Initial project plan was to develop an Image Caption Generator using a CNN and LSTM architecture as recommended by Bhawish Raj (BR)
 - **Key Features Developed:** GK (Gayathri Kumar) and HW (Henry Ward) researched NLP, while UOO (Uyi Owen-Ode) and BR researched CNNs & LSTM

- **Challenges and Solutions:** HW found NLP was an enormous topic not covered in this module but explained basics to the team. BR would ask lecturers if we can use a pre-trained CNN using transfer learning.
- **Sprint 2:** Each team members would attempt to create a rough working prototype of the Image Caption Generator
 - **Key Features Developed:** None
 - **Challenges and Solutions:** HW attempted but found that he could not create a working prototype and doubted it could be possible in the time available. Other team members agreed and decided to change the project proposal.
- **Sprint 3:** BR proposed to develop a simple chatbot as an alternative.
 - **Key Features Developed:** BR developed a working first prototype.
 - **Challenges and Solutions:** The chatbot prototype initially produced nonsensical responses so would require much improvement. However, after consultation with lecturers, we were told that such a chatbot would be out of scope for the module and we should develop a different project.
- **Sprint 4:** The team voted to develop a CNN that could classify melanomas after UOO found a suitable dataset.
 - **Key Features Developed:** 1. Preparation (Labeling) and Loading of Train and Test data (GK & HW) 2. Defined a basic model with Cross Validation (BR). Each team member was encouraged to develop models varying a specific hyperparameter: Regularisation (L1,L2) (GK), Dropout (CB), Early stopping (HW), Grid Search (UOO) Evaluation of models i.e. Plot graphs, confusion matrix, accuracy and loss (UOO & BR).
 - **Challenges and Solutions:** Many members found that training models took a long time using the University's Jupyter servers. HW purchased a Google Colab subscription in order to have access to GPU/TPUs to shorten training time.
- **Sprint 5:** The team decided to develop more models after learning from their previous models. Again, each team member trained models while varying specific hyperparameters
 - **Key Features Developed:** Early stopping (HW), remove layers, stride, output & hidden layer activation functions and filters (BR), Batchsize (32,64,128) (UOO), k-fold cross validation (GK), Batch Normalisation (CB)
 - **Challenges and Solutions:** Again, many members found that training models took a long time using the University's Jupyter servers. UOO decided to let his models train on the server overnight.

3. Testing and Feedback Loops:

- Formal continuous testing was not integrated into each sprint, such as unit, integration, and user acceptance testing (UAT). However, this could have improved our process as it could have ensured quality and adherence to requirements.
- Feedback from lecturers and fellow group members was solicited and incorporated into the development cycle, guiding future iterations.

4. Release and Review:

- **Deployment:** The final product, after rigorous testing and validation, was deployed to the production environment, in our case, simply model source code available for lecturers to assess.

- **Retrospective Meetings:** Post-deployment, the team conducted a retrospective meeting to discuss what went well, what could be improved, and how to implement these improvements in future projects. The findings of that retrospective are detailed in this report.

Conclusion

Utilizing Agile methodology, the development team was able to efficiently navigate the complexities of building a CNN for melanoma classification. The iterative approach fostered a highly adaptable and collaborative environment, enabling rapid responses to changing requirements and continuous improvement of the project. This development flow not only ensured the timely delivery of a robust and effective CNN model but also enhanced team dynamics and team and (hopefully) lecturer satisfaction.

Recommendations

- **Continuous Improvement:** Embrace the Agile principle of continuous improvement by regularly reviewing and refining development processes.
- **Stakeholder Engagement:** Maintain active engagement with stakeholders throughout the project to ensure alignment with user needs and project goals.
- **Skill Development:** Some team members were not familiar with some potential tools. We recommend investing in ongoing training and development for team members to stay abreast of the latest technologies and Agile practices, further enhancing project outcomes.

Conclusion (All/Available Person)

References

- Afroz, A. *et al.* (2023) 'Implementation of Convolutional Neural Networks deep learning approach to Classify Melanoma Skin Cancer', in *2023 Global Conference on Wireless and Optical Technologies (GCWOT). 2023 Global Conference on Wireless and Optical Technologies (GCWOT)*, pp. 1–7. Available at: <https://doi.org/10.1109/GCWOT57803.2023.10064671>.
- Cao, X. *et al.* (2021) 'Application of generated mask method based on Mask R-CNN in classification and detection of melanoma', *Computer Methods and Programs in Biomedicine*, 207, p. 106174. Available at: <https://doi.org/10.1016/j.cmpb.2021.106174>.
- Codella, N. *et al.* (2015) 'Deep Learning, Sparse Coding, and SVM for Melanoma Recognition in Dermoscopy Images', in L. Zhou *et al.* (eds) *Machine Learning in Medical Imaging*. Cham: Springer International Publishing, pp. 118–126. Available at: https://doi.org/10.1007/978-3-319-24888-2_15.
- Esteva, A. *et al.* (2017) 'Dermatologist-level classification of skin cancer with deep neural networks', *Nature*, 542(7639), pp. 115–118. Available at: <https://doi.org/10.1038/nature21056>.
- Forschner, A. *et al.* (2018) 'Diagnostic accuracy of dermatofluoroscopy in cutaneous melanoma detection: results of a prospective multicentre clinical study in 476 pigmented lesions', *British Journal of Dermatology*, 179(2), pp. 478–485. Available at: <https://doi.org/10.1111/bjd.16565>.
- Installing Python Packages from a Jupyter Notebook | Pythonic Perambulations* (no date). Available at: <https://jakevdp.github.io/blog/2017/12/05/installing-python-packages-from-jupyter/> (Accessed: 24 March 2024).
- Kawahara, J., BenTaieb, A. and Hamarneh, G. (2016) 'Deep features to classify skin lesions', in *2016 IEEE 13th International Symposium on Biomedical Imaging (ISBI). 2016 IEEE 13th International Symposium on Biomedical Imaging (ISBI)*, pp. 1397–1400. Available at: <https://doi.org/10.1109/ISBI.2016.7493528>.
- Kushimo, O.O. *et al.* (2023) 'Deep learning model to improve melanoma detection in people of color', *Arab Journal of Basic and Applied Sciences*, 30(1), pp. 92–102. Available at: <https://doi.org/10.1080/25765299.2023.2170066>.
- Murphree, D.H. *et al.* (2022) 'Deep Learning for Dermatologists: Part I Fundamental Concepts', *Journal of the American Academy of Dermatology*, 87(6), pp. 1343–1351. Available at: <https://doi.org/10.1016/j.jaad.2020.05.056>.
- Naeem, A. *et al.* (2020) 'Malignant Melanoma Classification Using Deep Learning: Datasets, Performance Measurements, Challenges and Opportunities', *IEEE Access*, 8, pp. 110575–110597. Available at: <https://doi.org/10.1109/ACCESS.2020.3001507>.
- Nasr-Esfahani, E. *et al.* (2016) 'Melanoma detection by analysis of clinical images using convolutional neural network', in *2016 38th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC). 2016 38th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)*, pp. 1373–1376. Available at: <https://doi.org/10.1109/EMBC.2016.7590963>.
- Pomponiu, V., Nejati, H. and Cheung, N.-M. (2016) 'Deepmole: Deep neural networks for skin mole lesion classification', in *2016 IEEE International Conference on Image Processing (ICIP). 2016 IEEE International Conference on Image Processing (ICIP)*, pp. 2623–2627. Available at: <https://doi.org/10.1109/ICIP.2016.7532834>.
- Robinson, J.K. *et al.* (2016) 'Early Detection of New Melanomas by Patients With Melanoma and Their Partners Using a Structured Skin Self-examination Skills Training Intervention: A Randomized Clinical Trial', *JAMA Dermatology*, 152(9), pp. 979–985. Available at: <https://doi.org/10.1001/jamadermatol.2016.1985>.
- Roetzheim, R.G. *et al.* (2000) 'Increasing supplies of dermatologists and family physicians are associated with earlier stage of melanoma detection', *Journal of the American Academy of Dermatology*, 43(2, Part 1), pp. 211–218. Available at: <https://doi.org/10.1067/mjd.2000.106242>.
- Stiff, K.M. *et al.* (2022) 'Artificial intelligence and melanoma: A comprehensive review of clinical,

dermoscopic, and histologic applications', *Pigment Cell & Melanoma Research*, 35(2), pp. 203–211. Available at: <https://doi.org/10.1111/pcmr.13027>.

Winkler, J.K. *et al.* (2019) 'Association Between Surgical Skin Markings in Dermoscopic Images and Diagnostic Performance of a Deep Learning Convolutional Neural Network for Melanoma Recognition', *JAMA Dermatology*, 155(10), pp. 1135–1141. Available at: <https://doi.org/10.1001/jamadermatol.2019.1735>.